UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS, MATEMATICAS Y FISICAS
CARRERA DE SOFTWARE

MATERIA:

GESTION DE LA CONFIGURACION DEL SOFTWARE

TEMA DE ACTIVIDAD:

DOCUMENTO 0: PLAN DEL PROYECTO DE SOFTWARE

(IEEE 1058)

DOCENTE:

ING. ALVAREZ SOLIS FRANCISCO XAVIER

ESTUDIANTE:

HUACON RAMIREZ GREGORY JOSUE

MALDONADO LOPEZ SAUL ALEJANDRO

ROMERO DIAZ ANDY ADRIAN

SOLIS SALAZAR ALEJANDRO SEBASTIAN

TENESACA PINCAY CINTHIA MARIA

CICLO ESCOLAR:

2025 – 2026 CICLO II

Contenido

1. Introducción y Alcance del Proyecto 1

1.1 Objetivo del Proyecto 1

1.2 Alcance del Sistema (Entregables de Software) 1

2. Organización del Equipo de Desarrollo 2

3. Metodología de Gestión del Desarrollo 3

3.1 Modelo de Proceso: Metodología Ágil – Scrum 3

3.2 Herramientas de Desarrollo y Gestión 3

4. Planificación del Cronograma (Paquetes de Trabajo) 3

4.1. Fase de Inicio y Arquitectura 3

4.2. Sprint 1: Núcleo del Sistema 4

4.3. Sprint 2: Gestión de Reparaciones 4

4.5. Fase de Despliegue y Pruebas Finales 4

5. Plan de Calidad y Gestión de Riesgos Técnicos 4

5.1 Estrategia de Calidad 4

5.2 Gestión de Riesgos 4

6. Plan de Control de Cambios (Técnico) 5

1. Introducción y Alcance del Proyecto

1.1 Objetivo del Proyecto

El objetivo principal es desarrollar, probar e implementar una aplicación web transaccional que automatice íntegramente el ciclo de negocio del Taller de Celulares. El sistema permitirá registrar órdenes de servicio, controlar inventario, manejar información de los clientes y emitir documentación tributaria.

Además, la solución busca:

- Eliminar el seguimiento manual de reparaciones, reduciendo errores humanos.
- Garantizar trazabilidad desde la recepción del dispositivo hasta la entrega final.
- Aumentar la eficiencia operativa mediante alertas, reportes y paneles de control.
- Ofrecer una experiencia más transparente al cliente mediante un portal de consulta.

1.2 Alcance del Sistema (Entregables de Software)

El software incluirá los siguientes módulos, cada uno con funcionalidades específicas:

1. **Módulo de Taller**
   - Registro de órdenes de servicio con datos del cliente y descripción del equipo.
   - Captura de diagnóstico preliminar y costos estimados.
   - Seguimiento del estado de la reparación en tiempo real.
   - Historial de reparaciones por IMEI o número de serie.
2. **Módulo de Inventario**
   - Gestión de repuestos: ingreso, salida, ajustes y devoluciones.
   - Control de stock crítico mediante alertas automáticas.
   - Registro de proveedores y costos unitarios.
   - Reportes de movimiento de inventario y valorización.
3. **Módulo de Cliente (Portal Web)**
   - Acceso seguro mediante código o número de orden.
   - Consulta del avance del trabajo (porcentaje, estado, fecha estimada).

- Notificaciones automáticas por correo/SMS cuando el equipo esté listo.

4. **Módulo de Facturación**
    - Generación de facturas electrónicas o comprobantes físicos.
    - Cálculo automático de impuestos y totales.
    - Reportes financieros: ingresos por día, mes o técnico.
    - Integración futura con sistemas contables externos (API opcional).

2. Organización del Equipo de Desarrollo

El proyecto se llevará a cabo mediante un equipo con roles especializados para asegurar calidad, tiempos y cumplimiento técnico.

1. **Líder Técnico (Tech Lead)**
    - Define la arquitectura del sistema (Frontend, Backend, Base de Datos, Servicios).
    - Selecciona tecnologías, librerías y estándares de desarrollo.
    - Supervisa integración, revisiones de código y decisiones críticas.
    - Asegura buenas prácticas de escalabilidad y seguridad.
2. **Desarrollador Frontend (React Specialist)**
    - Construcción de interfaces responsivas con React y Tailwind/Material UI.
    - Implementación de flujos de usuario, validaciones y UI/UX.
    - Consumo de APIs REST y manejo de estados (Redux, Zustand, Context).
    - Garantiza compatibilidad con navegadores y dispositivos móviles.
3. **Desarrollador Backend (Node Specialist)**
    - Creación de APIs RESTful con Express o Nest.js.
    - Modelado de datos y comunicación con la base de datos (ORM: Prisma/Sequelize).
    - Implementación de autenticación, autorización y rendimiento.
    - Gestión de logs, auditoría y seguridad (JWT, CORS, rate limiting).
4. **QA Tester (Aseguramiento de Calidad)**
    - Elaboración del plan de pruebas y casos de prueba.
    - Validación funcional, pruebas de regresión y pruebas automatizadas.
    - Registro de incidencias y seguimiento hasta su resolución.
    - Certificación previa al despliegue.

3. Metodología de Gestión del Desarrollo

3.1 Modelo de Proceso: Metodología Ágil – Scrum

Se trabajará con ciclos iterativos de **2 semanas**, entregando incrementos funcionales al cliente.

Componentes Clave del Proceso

- **Product Backlog:** Contiene todas las funcionalidades del sistema como historias de usuario, priorizadas por valor y complejidad.
- **Sprint Planning:** El equipo selecciona qué tareas se desarrollarán en el sprint, basadas en capacidad y prioridades.
- **Dailies:** Reuniones de 15 min donde se revisa qué se hizo, qué se hará y qué bloqueos existen.
- **Sprint Review:** Demostración del avance al cliente al finalizar el sprint para obtener retroalimentación.
- **Sprint Retrospective:** Revisión interna del equipo para identificar mejoras en el proceso.

3.2 Herramientas de Desarrollo y Gestión

- **Control de Código – Git/GitHub o GitLab:** Se aplicará Gitflow con ramas: *main*, *develop*, *feature/*, *release/*, *hotfix/*.
- **Gestión de Tareas – Jira/Trello:** Uso de tablero Kanban para visualizar progreso y estados.
- **Entorno de Desarrollo – VS Code o WebStorm:** Configuración de linters, formateadores y extensiones de productividad.
- **Pruebas de API – Postman o Insomnia:** Creación de colecciones de pruebas automatizadas, documentación y mock servers.

4. Planificación del Cronograma (Paquetes de Trabajo)

4.1. Fase de Inicio y Arquitectura

- Configuración del repositorio, entorno React + Node, dependencias y scripts.
- Diseño del modelo entidad–relación para base de datos relacional o NoSQL.
- Definición de arquitectura por capas (Controllers, Services, Repositories).
- Configuración inicial de CI/CD.

4.2. Sprint 1: Núcleo del Sistema

- Implementación del sistema de autenticación con roles: Admin, Técnico, Cliente.
- CRUD de inventario: repuestos, categorías, stock y costos.
- Pruebas unitarias de endpoints básicos.

4.3. Sprint 2: Gestión de Reparaciones

- Formulario de recepción del dispositivo con datos obligatorios.
- Lógica de diagnóstico técnico: fallas, pruebas y repuestos requeridos.
- Flujo completo de estados: **Recibido → En Revisión → En Reparación → Listo para Entrega**.
- Dashboard para técnicos con tareas asignadas.

4.4. Sprint 3: Portal Cliente y Facturación

- Construcción de panel público para clientes.
- Integración con módulo de reparaciones para consultar estados.
- Implementación de facturación con impuestos y descuentos.
- Reportes PDF/Excel (Ingresos, Inventario, Técnicos).

4.5. Fase de Despliegue y Pruebas Finales

- Pruebas de carga y stress testing (Locust, JMeter).
- Integración completa entre módulos.
- Despliegue en hosting o cloud (AWS, Vercel, Render).
- Capacitación al cliente final y entrega de manuales.

5. Plan de Calidad y Gestión de Riesgos Técnicos

5.1 Estrategia de Calidad

- **Code Reviews:** Revisión obligatoria para cada *pull request* antes de unir código a develop o main.
- **Pruebas Unitarias:** Uso de Jest, Vitest o Mocha para funciones críticas (cálculo de totales, stock).
- **Estándares de Código:** ESLint + Prettier en ambos (React y Node). Uso de patrones de diseño: MVC, servicios, repositorios.
- **Pruebas Automatizadas de UI:** Cypress o Playwright opcional para flujos principales.

5.2 Gestión de Riesgos

1. **Riesgo: Inconsistencia de datos entre Frontend y Backend**
   Mitigación: Definir el contrato de API (Swagger/OpenAPI), versionamiento de la API.
2. **Riesgo: Consultas lentas o bloqueos en inventario**
   Mitigación: Indexación, caching en servidor, paginación en interfaces.
3. **Riesgo: Vulnerabilidades de seguridad (SQL Injection, XSS, CSRF)**
   Mitigación: Uso de ORM, sanitización, validación con Yup/Joi, uso de HTTPS.
4. **Riesgo: Fallos en despliegue o rollback difícil**
   Mitigación: Implementar CI/CD con pruebas antes de desplegar y versiones etiquetadas.

6. Plan de Control de Cambios (Técnico)

1. Registrar el cambio en el sistema de gestión (Trello/Jira) con prioridad e impacto.
2. El Líder Técnico evalúa:
   - Riesgo técnico.
   - Impacto en arquitectura, base de datos y módulos afectados.
   - Tiempo estimado de implementación.
3. Si el cambio es menor, se ejecuta en el mismo sprint; si implica:
   - refactorización extensa,
   - cambio estructural,
   - migración de base de datos, entonces se planifica para el siguiente sprint.